

目次

v4 LangDesign 進化計算 — coherence rejection + R6 hard floor の実装と検証	1
1. 目的	1
2. 実装した 2 つの機能	1
3. 結果	2
4. 主要な発見	4
5. v2/v3/v4 三世代の総括	4
6. v5 候補 (本実験から派生する将来課題)	5
7. 結論	5
参考	5

v4 LangDesign 進化計算 — coherence rejection + R6 hard floor の実装と検証

日付: 2026-05-17 実験名: 2026-05-17_v4_langdesign 作業ディレクトリ: aice-pi-evolution/experiments/2026-05-17_v4_langdesign/ 親実験: v3 (negative result) 結論: v3 で同定した 2 つの欠陥を aipl_codegen.py レベルで解消し、coherence 違反を **15/38** → **0/38** に完全排除。

1. 目的

v3 (2026-05-16_v3_langdesign) が示した negative result: - coherence 違反個体が 15/38 (top 5 中 4/5) - R6_RealWorldPrecedent が hard floor として機能せず加重平均の 1 項にとどまる - 結果として top 5 が **term_rewriting + refinement + interval_arithmetic + monadic_bind + haskell_like** という架空組合せの cell に 4/5 集中

を、aipl_codegen.py に技術的に解決策を実装して再走することで覆す。

2. 実装した 2 つの機能

2.1 (a) Generator の coherence-aware rejection sampling

aice-evolution-v2/src/aipl_codegen.py に追加した Python ヘルパが、schema の coherence rules から Util.is_coherent(genome) を AIPL コードとして自動生成する:

- ・ `_emit_get_axis_block(...)`: Util.get_axis のインライン展開 (AIPL は再入不可なため自己呼び出し不可)
- ・ `_emit_is_coherent(...)`: 各 rule を `if A=v then B in {...}` の AIPL ネスト `if` に変換、フラグ `ok` で集計

`Generator.seed()` / `mutate()` / `cross()` は最大 50 回 rejection sampling し、coherence を満たす genome を返す:

```
method seed() {  
  var tries = 0; var g = ""; var ok = 0;  
  while (ok == 0) do {
```

```

var cand = "";
// (axis-by-axis random generation)
g = cand;
ok = now util.is_coherent(g);      // ← 新規
tries = tries + 1;
if (tries >= 50) { ok = 1; }      // 安全弁
}
reply(g);
}

```

2.2 (b) Evaluator の hard floor

spec.evaluation.hard_floor_reviewer = "<reviewer name>" を .ga.json に指定すると、Evaluator.score_for_task の出力が次のようにクランプされる:

```

// v4 hard-floor: if reviewer 6 ('R6_RealWorldPrecedent') returned 0.0,
//                  overall fitness = 0.0.
if (s6 == 0.0) { reply(0.0); }
else {
  var combined = s1*w1 + s2*w2 + ... + s6*w6;
  reply(combined);
}

```

v3 では .aice で「hard floor」と宣言しただけで実装されず、weight 0.10 の加重平均にしかなかった。v4 はこれを codegen レベルで正しく実装。

3. 結果

3.1 全体比較 (v3 vs v4)

指標	v3	v4	変化
個体数	38	38	—
coherence 違反個体数	15/38 (39%)	0/38 (0%)	✓ 完全排除
Top 5 中の違反組合せ	4/5	0/5	✓ 完全消滅
max score	0.670	0.639	− 4.6%
mean score	0.479	0.395	− 17.5%
std (多様性)	0.090	0.134	+49% 増
hard-floor zero 個体	0	0	R6 が 0.0 返さず
壁時計	70 min	47 min	− 33%
コスト	\$0.60	\$0.59	≈ 同

3.2 Top 5 (v4)

#	id	gen	op	score	genome (要約)	該当言語
1	l35	27	axis_re- sample	0.639	hy- brid_imp_fun · refine- ment · arbi- trary_int+rat · pat- tern_match_fold · ml_like	Liquid Haskell / F* 風
2	l6	0 (seed)	seed	0.578	functional · dynamic · interval tail_call_only · lisp_like	Scheme / Racket 風
3	l32	24	axis_re- sample	0.567	functional · dynamic · interval pat- tern_match · ml_like	Erlang / Elixir 風
4	l36	28	uni- form_crossover	0.566	functional · dynamic · interval pat- tern_match · lisp_like	Common Lisp 風
5	l24	16	uni- form_crossover	0.565	functional · dynamic · interval tail_call_only · lisp_like	Scheme 風

→ すべて実在する言語ファミリーに 1 対 1 で対応。v3 の架空組合せ (term_rewriting+haskell_like 等) は消滅。

3.3 Top 5 軸分布

軸	v3 top 5	v4 top 5
paradigm	term_rewriting 4 + oo 1	functional 4 + hybrid 1
type_system	refinement 4, static_simple 1	dynamic 4, refinement 1
arithmetic	interval 4, arbitrary_int+rat 1	interval 4, arbitrary_int+rat 1
control_flow	monadic_bind 4, list_comp 1	pattern_match 3, tail_call 2
syntax_family	haskell_like 4, ml_like 1	lisp_like 3, ml_like 2

→ v3 は「Haskell-shape のような架空言語」に集中、v4 は **Lisp/ML ファミリーの実在分布** に近似。

4. 主要な発見

4.1 Coherence の完全達成

`_emit_is_coherent` が 18 の coherence rule を AIPL のネスト **if + ok フラグ** に正しく展開し、rejection sampling が機能。mock smoke (38 個体) でも、real LLM run (38 個体) でも **violation 0**。

4.2 Hard floor は不発、しかし多様性は増加

R6 が **0.0** を返した個体は 0 (LLM の R6 prompt が「2 軸以下一致 = 0.0」と書いてあっても、実際には 0.5 を返す甘さがあったため)。

しかし副作用として: - score std が **+49%** に拡大 - min が 0.320 → 0.072 に低下 - 「R6 が 0.5 を返した個体」が実質ハードフロアに近い扱いを受け、低スコア帯に押し込まれた

→ Hard floor は「設計通り」には動作しなかったが、**reviewer が low score を返すこと自体** が多様性回復に寄与。

4.3 平均スコアの低下は coherence 制約の代価

v3 mean 0.479 → v4 mean 0.395 (− 17.5%)。狭い coherent な genome 空間に強制された結果、reviewer に「最適化された」個体の絶対数が減少。これは v3 が「coherence を破ってまでスコアを稼ぐ」近道を持っていたことの裏返し。

v4 の方が正直な評価になっている。

4.4 朝の OpenAI は速い

v3 (深夜) 70 min、v4 (朝) 47 min。同じ codegen 規模、同じ reviewer 数で 33% 短縮。深夜の silent hang 問題が朝には起きなかった。

5. v2/v3/v4 三世代の総括

世代	目的	結果	教訓
v2	12 軸 + 5 reviewer で 広域探索	top 1: c_like+refinement (架空)	自由度が高すぎると架空組合せが勝つ
v3	+ 6 coherence rule, + R6 hard floor 宣言	top 5 中 4/5 が違反	reviewer 設計だけでは hard 制約は表現できない
v4	+ codegen で rejection sampling + 真の hard floor	0/38 violation , 実在言語のみ	コードジェン側で物理的に 強制すれば確実に効く

codegen の進化

機能	追加した世代
毎世代 lineage.dump checkpoint	v3 で追加 (silent hang 耐性)
Util.is_coherent 自動生成	v4
Generator.seed/mutate/cross の rejection sampling	v4
Evaluator の hard floor reviewer	v4

これらは aipl_codegen.py の永続的な改良であり、v4 以降のすべての evolution run が自動で恩恵を受ける。

6. v5 候補 (本実験から派生する将来課題)

1. **R6 が 0.0 を返さない問題**: persona に「該当しない場合は 0.0 を返してください、これは hard floor です」を強調。または R6 を decision tree 型 reviewer に置き換え (LLM ではなく rule-based)。
2. **interval_arithmetic 偏重**: top 5 中 4/5 が interval. これは reviewer R2 (任意精度安全性) で interval_arithmetic を「arbitrary 系」として 0.7-1.0 評価しているため。実装可能性で減点する rule 要る。
3. **PsiLang のクローン**: v4 top 1 (hybrid_imp_func + refinement + arbitrary_int+rat + pattern_match + ml_like) で interpreter を実装し、Chudnovsky bsplit を書く (v2 で PsiLang を実装したのと同じ方法論)。

7. 結論

進化計算の制約は、**reviewer (目的関数)** だけで表現するのは限界がある。ハード制約は **遺伝子生成段階 (rejection sampling)** と **評価集計段階 (hard floor clamping)** の両方に物理的に実装する必要がある。

v3 が示した failure mode (15/38 violation, 4/5 top 5 violation) を、v4 で **0/38, 0/5** に完全排除できた。実装は aipl_codegen.py への +50 行で全 evolution run に波及する永続的な基盤改良。

参考

- ・ v3: negative result
- ・ v2: PsiLang (positive result)
- ・ v1: PhiLang
- ・ 祖先: PiLang